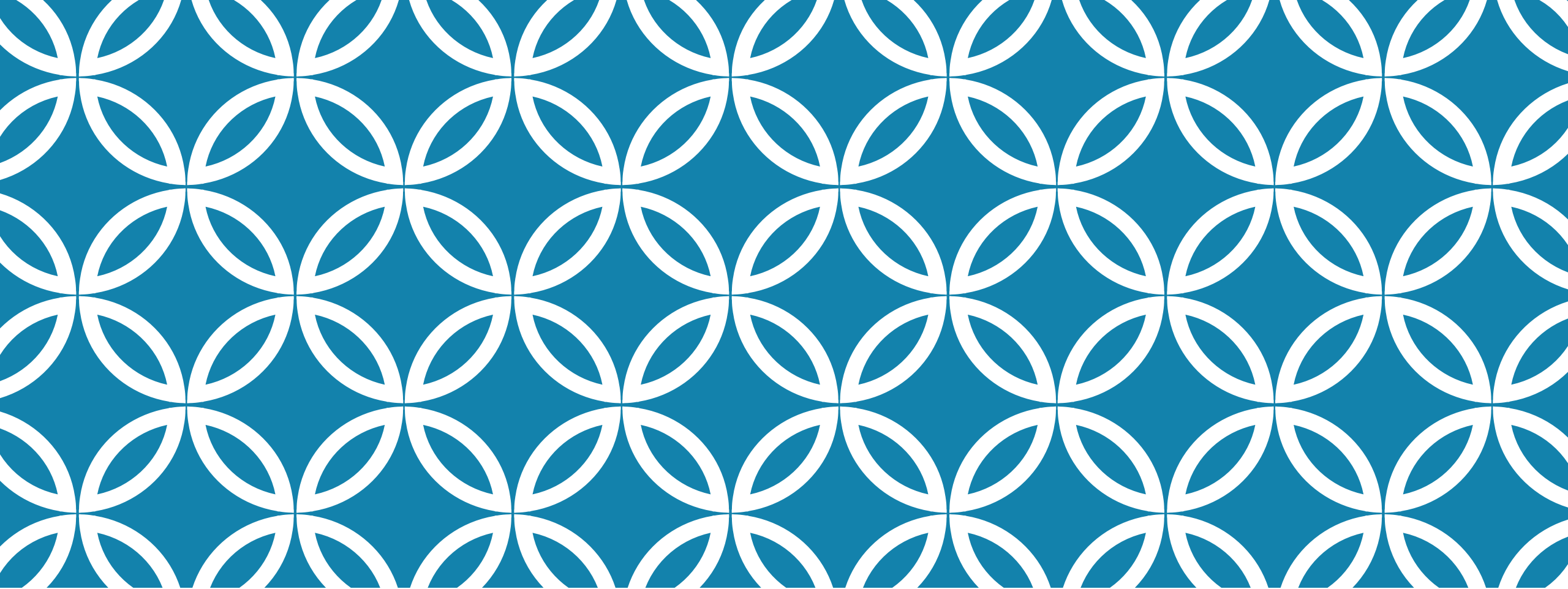




Computer Graphics (3)

Global Variables

```
#include<GL/glut.h>
```

```
#include <stdio.h>
```

```
int phyWidth= 700;
```

```
int phyHeight= 700;
```

```
int logWidth=100;
```

```
int logHeight=100;
```

```
int centerX=logWidth/2;
```

```
int centerY=logHeight/2;
```

- Physical width and height of the window

- Logical width and height of the window

Main()

```
int main( int argc, char ** argv)
{
    glutInit( &argc, argv);
    glutInitDisplayMode( GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowPosition( 100, 100);
    glutInitWindowSize(phyWidth, phyHeight);
    glutCreateWindow( "problem demonstration");
    init2D();
    glutDisplayFunc(Display);
    glutMainLoop();
}
```

```
void init2D()
{
    glClearColor( 0.0, 0.0, 1.0, 1.0); // COLOR
                                         BACKGROUND
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D( 0.0, logWidth, 0.0, logHeight);
}

void Display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    // .....
    glutSwapBuffers();
    glFlush();
}
```

POINTS

```
glBegin(GL_POINTS);
```

```
    glVertex2i(50,50);
```

```
    glVertex2i(50,60);
```

```
    glVertex2i(60,60);
```



Takes N points

```
glEnd();
```

LINES

```
glBegin(GL_LINES);
```

```
    glVertex2i(0,0);
```

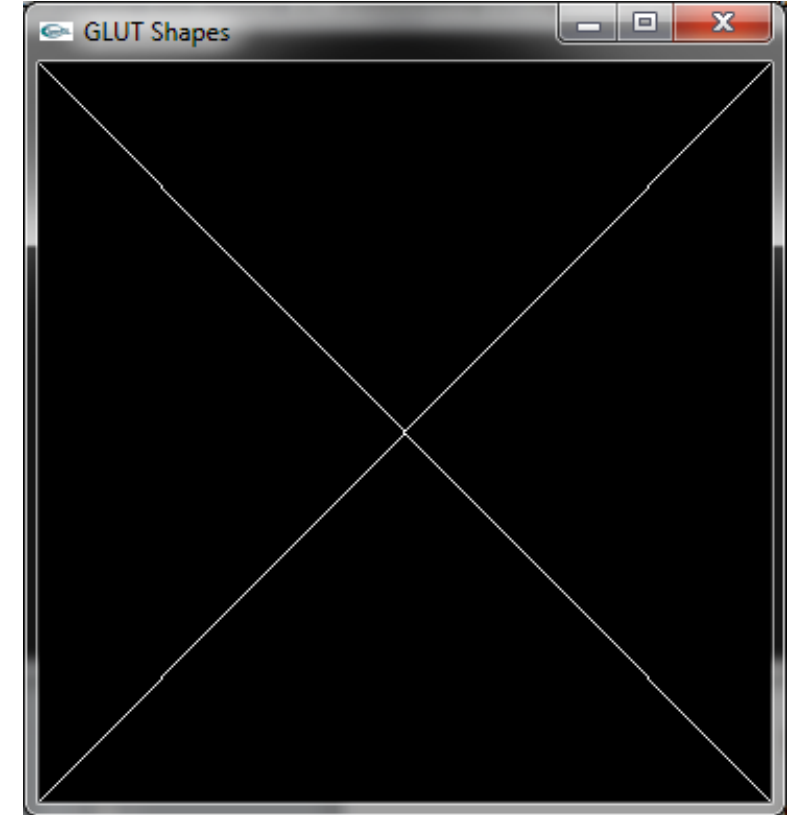
```
    glVertex2i(100,100);
```

```
    glVertex2i(100,0);
```

```
    glVertex2i(0,100);
```

Draws N Lines;
2N vertices

```
glEnd();
```



LINE STRIP

```
glBegin(GL_LINE_STRIP);
```

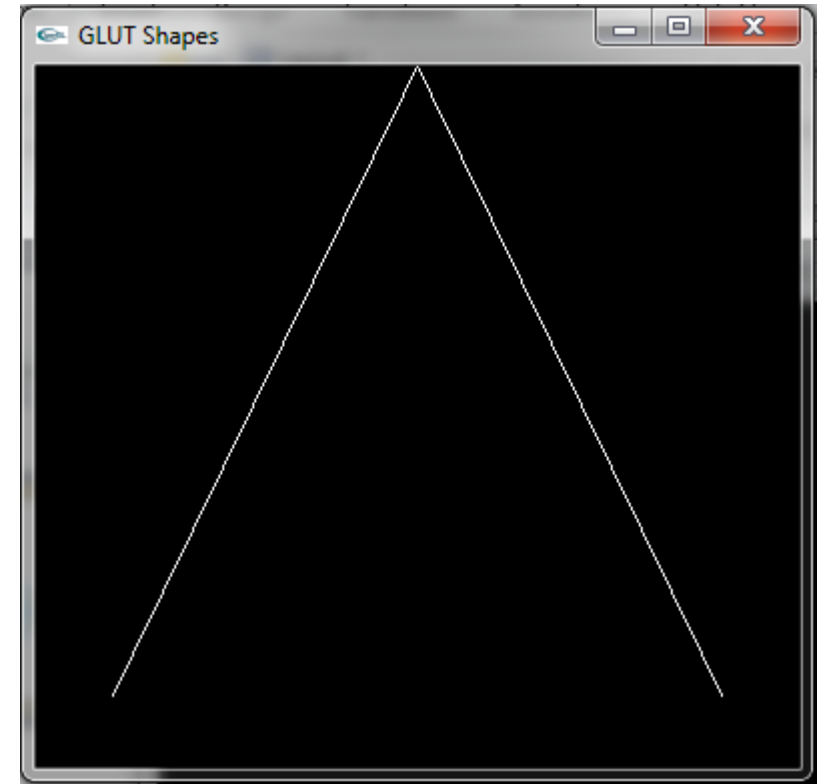
```
glVertex2i(10,10);
```

```
glVertex2i(50,100);
```

```
glVertex2i(90,10);
```

```
glEnd();
```

Draws N
connected
lines from the
first vertex to
the last



LINE LOOP

```
glBegin(GL_LINE_LOOP);
```

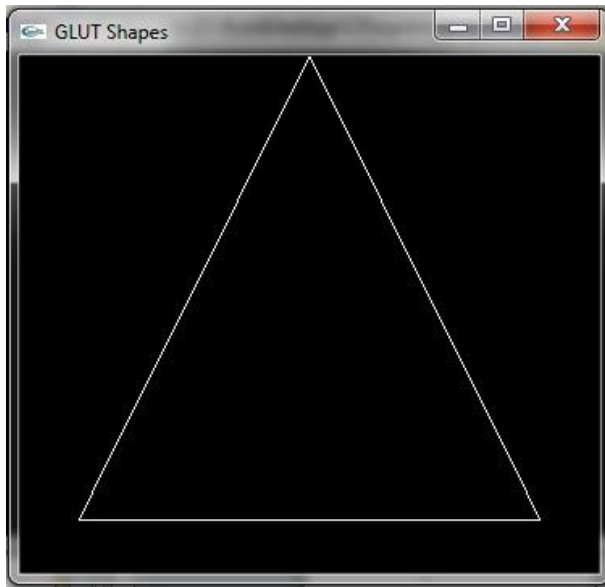
```
    glVertex2i(10,10);
```

```
    glVertex2i(50,100);
```

```
    glVertex2i(90,10);
```

```
glEnd();
```

Draws N
connected
lines from the
first vertex to
the last, then
back to the
first.



```
glBegin(GL_LINE_LOOP);
```

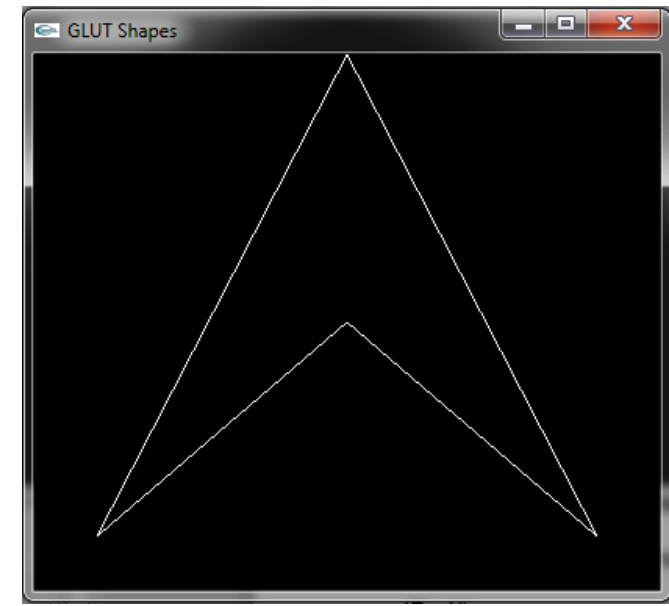
```
    glVertex2i(10,10);
```

```
    glVertex2i(50,100);
```

```
    glVertex2i(90,10);
```

```
    glVertex2i(50,50);
```

```
glEnd();
```



TRIANGLES

```
glBegin(GL_TRIANGLES);
```

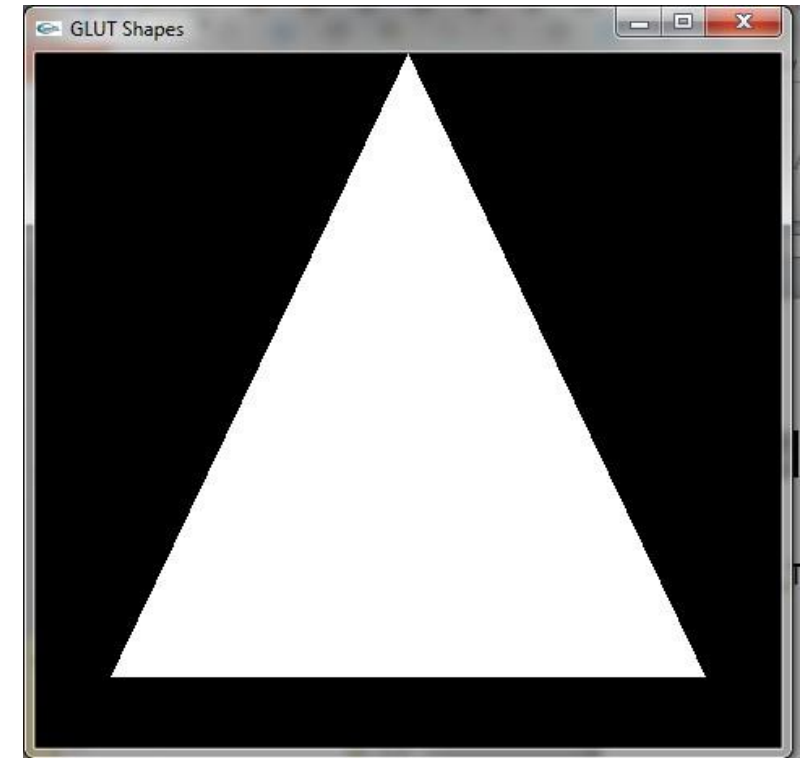
```
    glVertex2i(10,10);
```

```
    glVertex2i(50,100);
```

```
    glVertex2i(90,10);
```

```
glEnd();
```

Treats each
triplet of
vertices as an
independent
triangle.



QUADS

```
glBegin(GL_QUADS);
```

```
    glVertex2i(10,10);
```

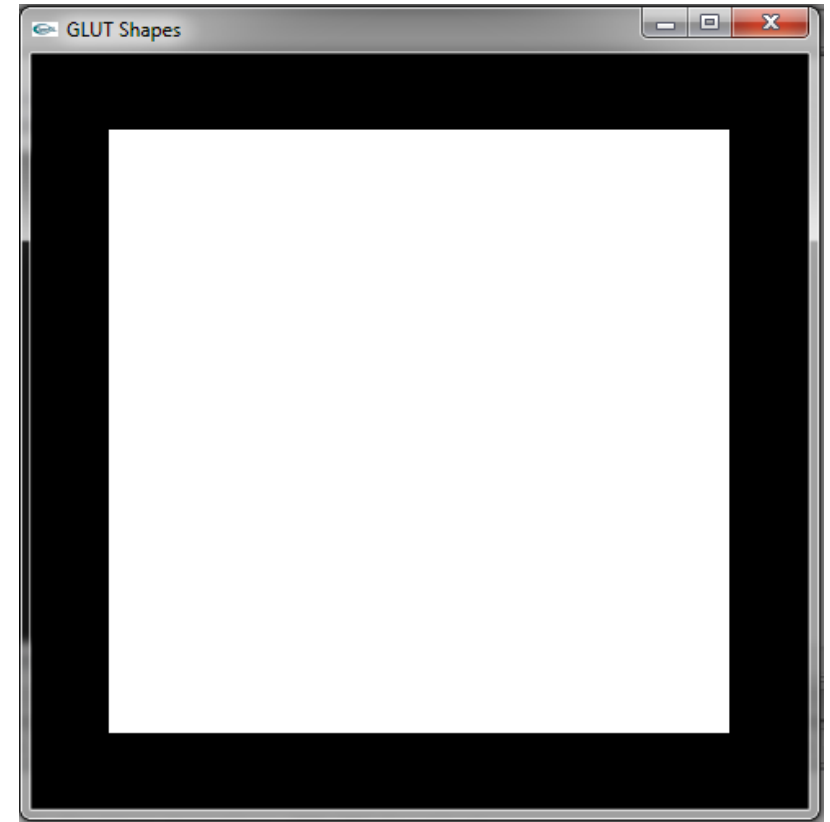
```
    glVertex2i(10,90);
```

```
    glVertex2i(90,90);
```

```
    glVertex2i(90,10);
```

Treats each
group of four
vertices as an
independent
quadrilateral

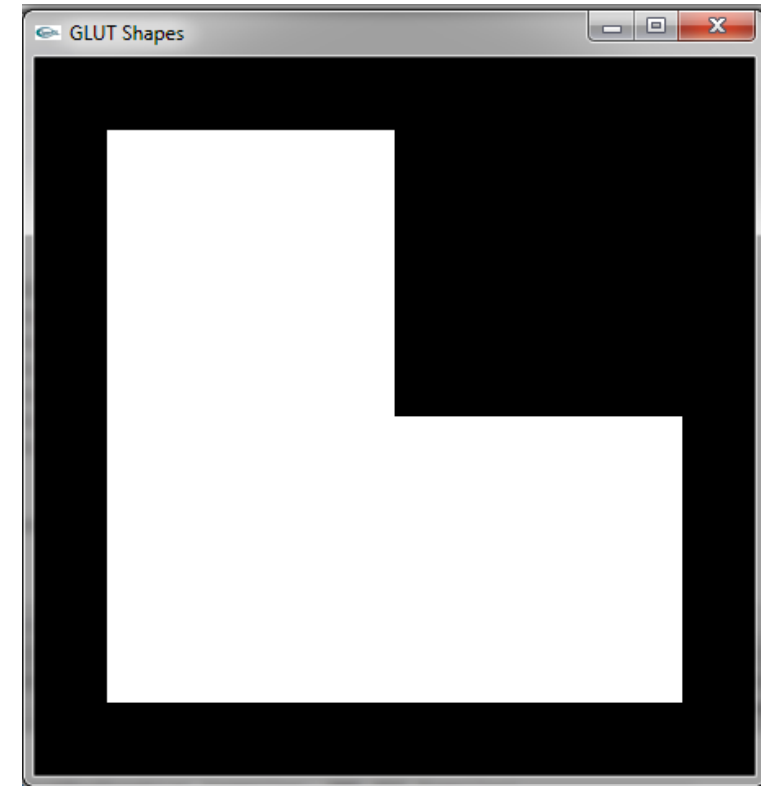
```
glEnd();
```



POLYGON

```
glBegin(GL_POLYGON);  
    glVertex2i(10,10);  
    glVertex2i(10,90);  
    glVertex2i(50,90);  
    glVertex2i(50,50);  
    glVertex2i(90,50);  
    glVertex2i(90,10);  
  
glEnd();
```

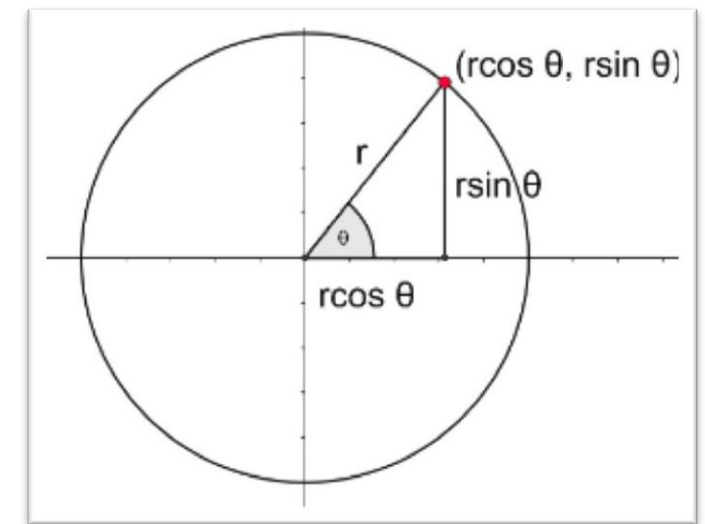
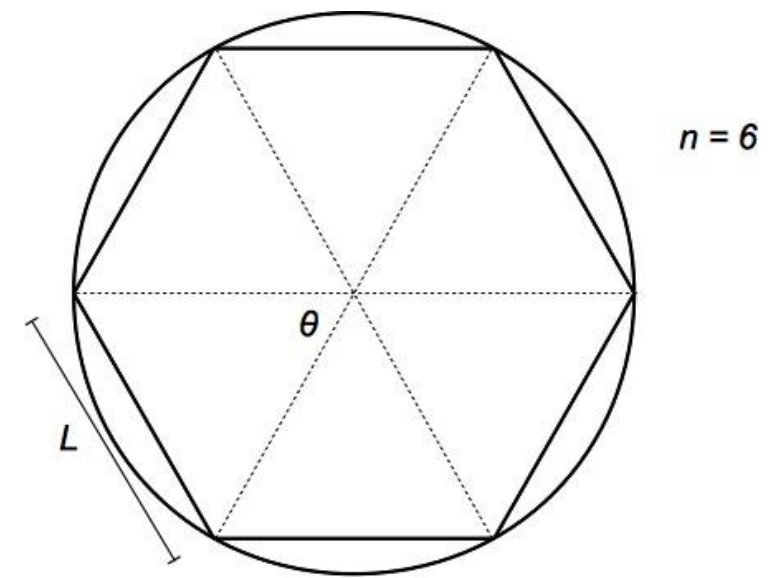
Draws a single polygon.



CIRCLE

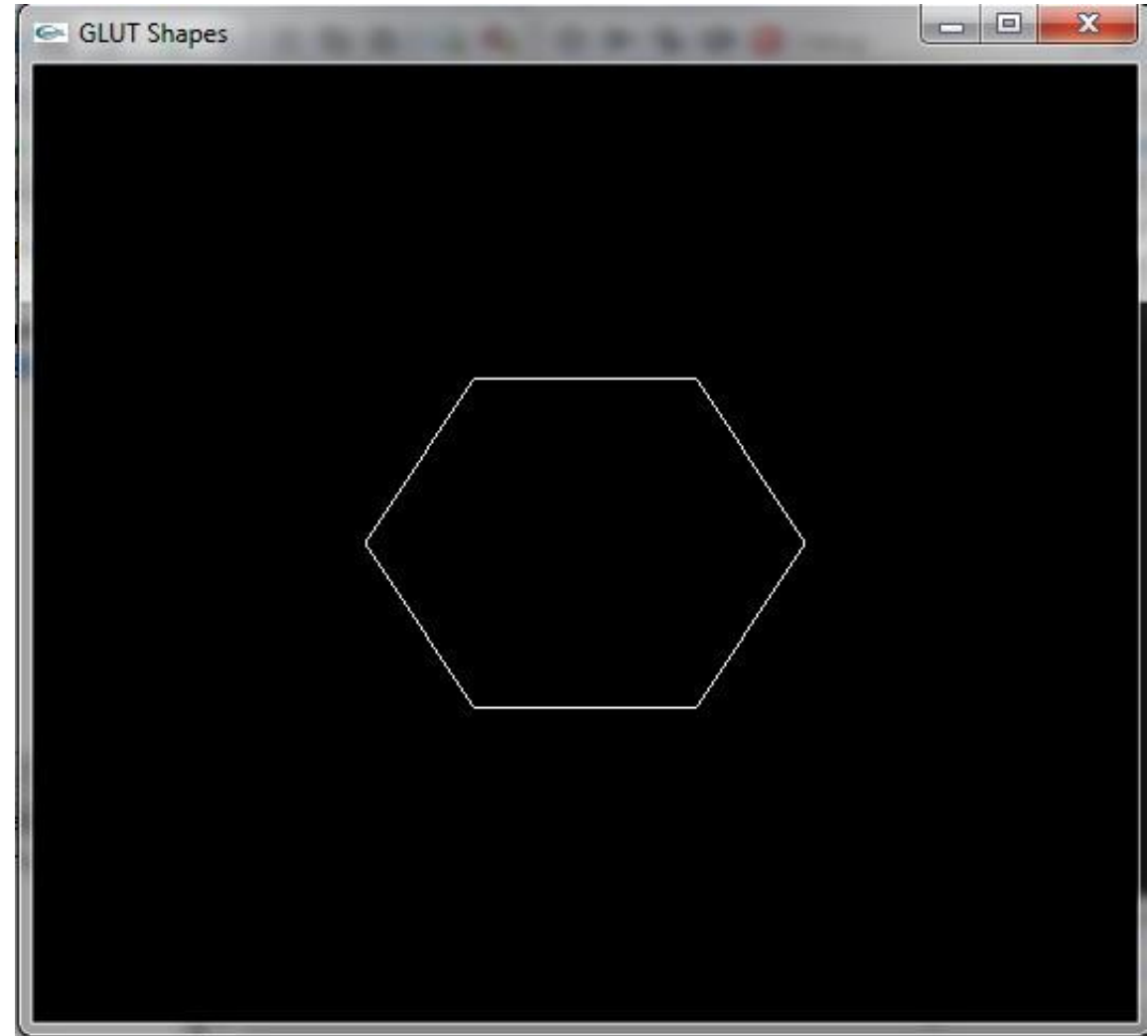
```
void DrawCircle(float cx, float cy, float r, int num_segments) {  
    glBegin(GL_LINE_LOOP);  
    for (int i = 0; i < num_segments; i++) {  
        float theta = 2.0f * 3.1415926f * float(i) / float(num_segments); //get the current angle in radians  
        float x = r * cosf(theta); //calculate the x component  
        float y = r * sinf(theta); //calculate the y component  
        glVertex2f(x + cx, y + cy); //output vertex  
    }  
    glEnd();  
}
```

N.B.: #include <math.h>



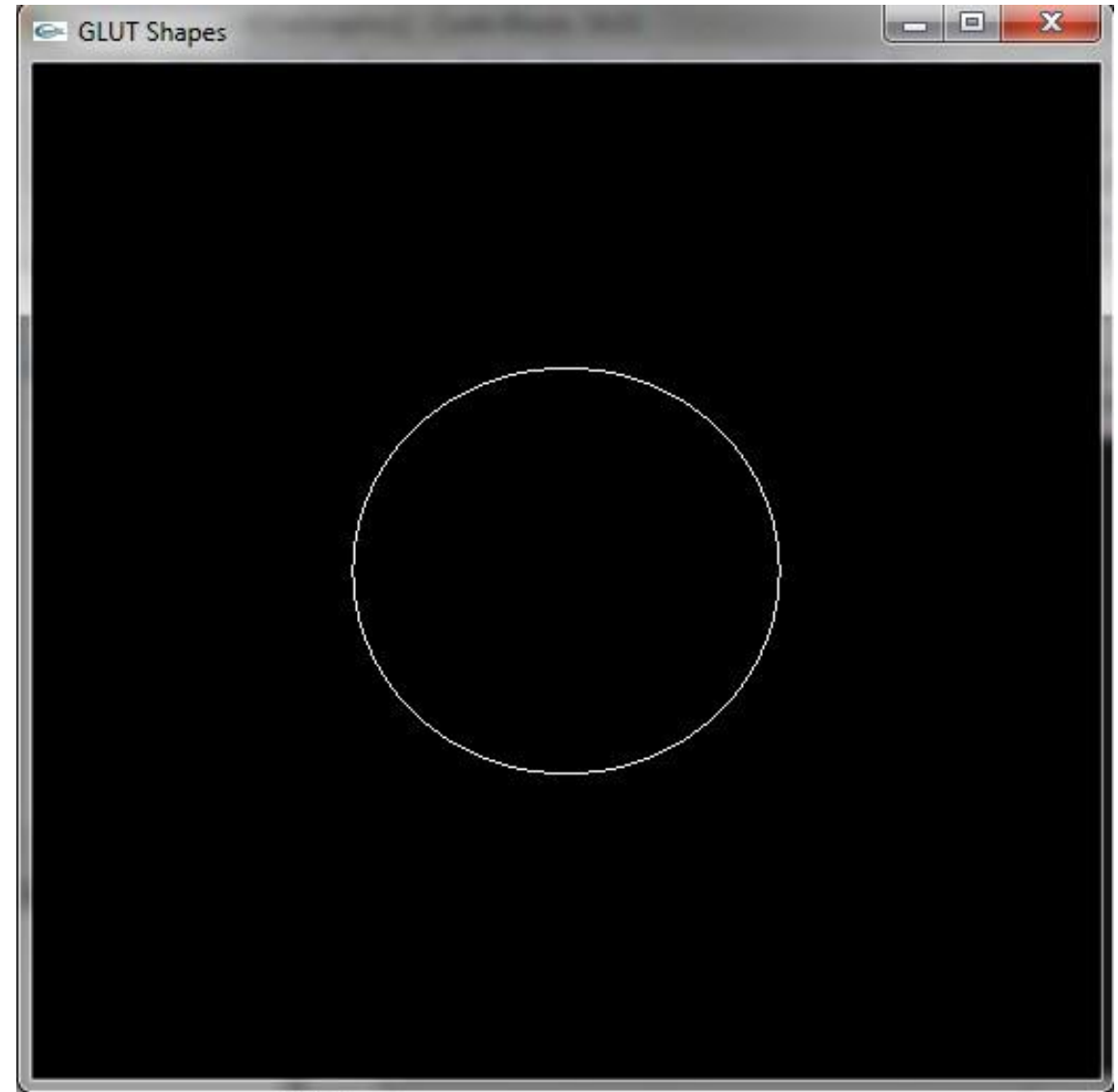
DISPLAY

```
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    DrawCircle(centerX, centerY, 20, 6);
    glutSwapBuffers();
    glFlush();
}
```



DISPLAY

```
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    DrawCircle(centerX, centerY, 20, 100);
    glutSwapBuffers();
    glFlush();
}
```



CIRCLE (SOLID)

```
void DrawCircle(float cx, float cy, float r, int num_segments)
{
    glBegin(GL_POLYGON);
    for (int i = 0; i < num_segments; i++) {
        float theta = 2.0f * 3.1415926f * float(i) / float(num_segments); //get the current angle
        float x = r * cosf(theta); //calculate the x component
        float y = r * sinf(theta); //calculate the y component
        glVertex2f(x + cx, y + cy); //output vertex
    }
    glEnd();
}
```

N.B.: #include <math.h>

CIRCLE

Do you remember DDA?

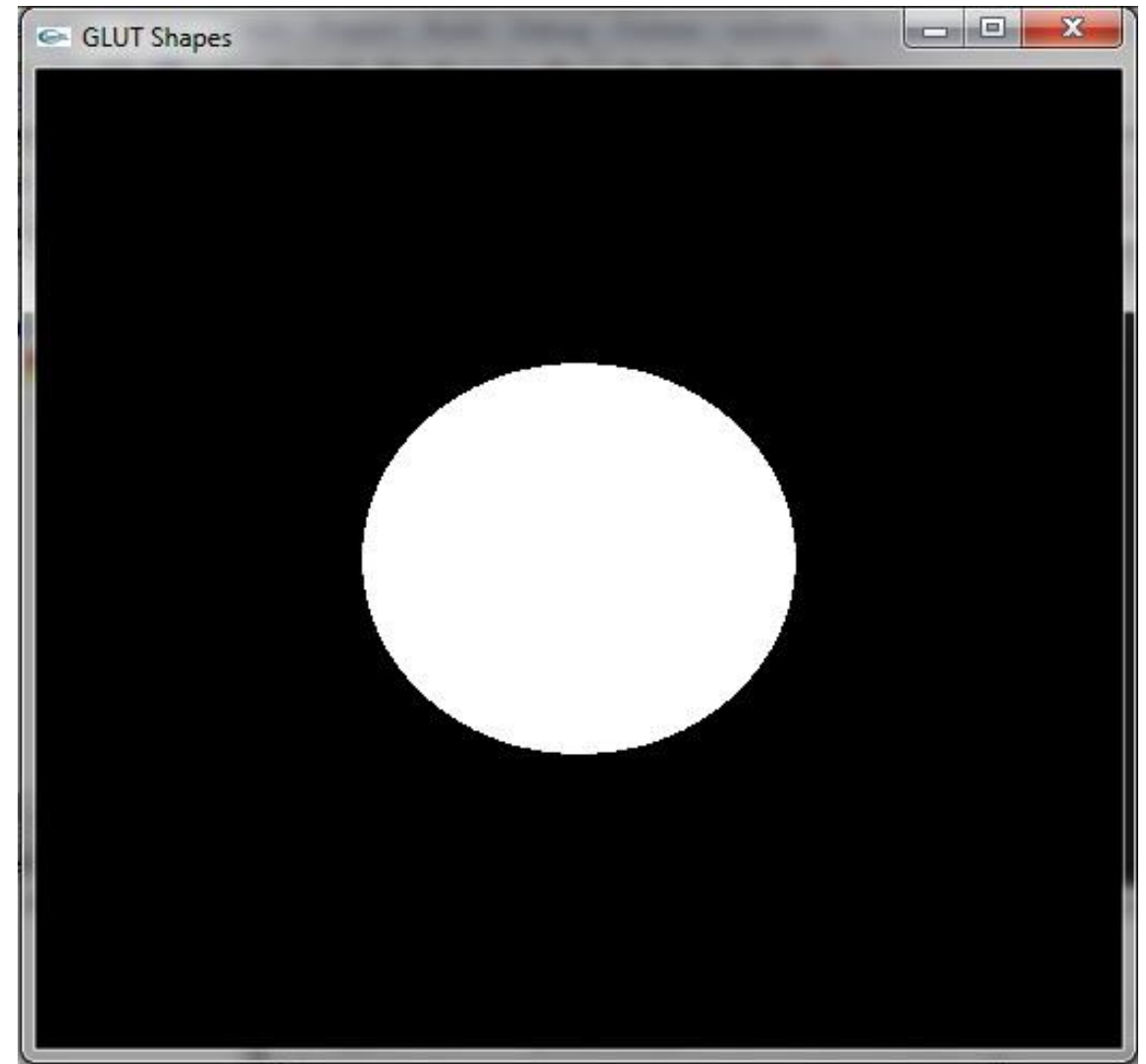
- It was bad for dealing with floats
- The same for the previous approach

MID-POINT CIRCLE ALGORITHM - STEPS

1. Input radius r and circle center (x_c, y_c) and Set the first point $(x_0, y_0) = (0, r)$.
2. Calculate the initial value of the decision parameter as $p_0 = 1 - r$.
3. At each x_k position, starting at $k = 0$, perform the following test:
If $p_k < 0$, plot $(x_k + 1, y_k)$ and $p_{k+1} = p_k + 2x_{k+1} + 1$,
Otherwise,
plot $(x_k + 1, y_k - 1)$ and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$,
4. Determine symmetry points on the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values: $x = x + x_c, y = y + y_c$
6. Repeat steps 3 though 5 until $x \geq y$.
7. For all points, add the center point (x_c, y_c)

DISPLAY

```
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    DrawCircle(centerX,centerY,20,100);
    glutSwapBuffers();
    glFlush();
}
```



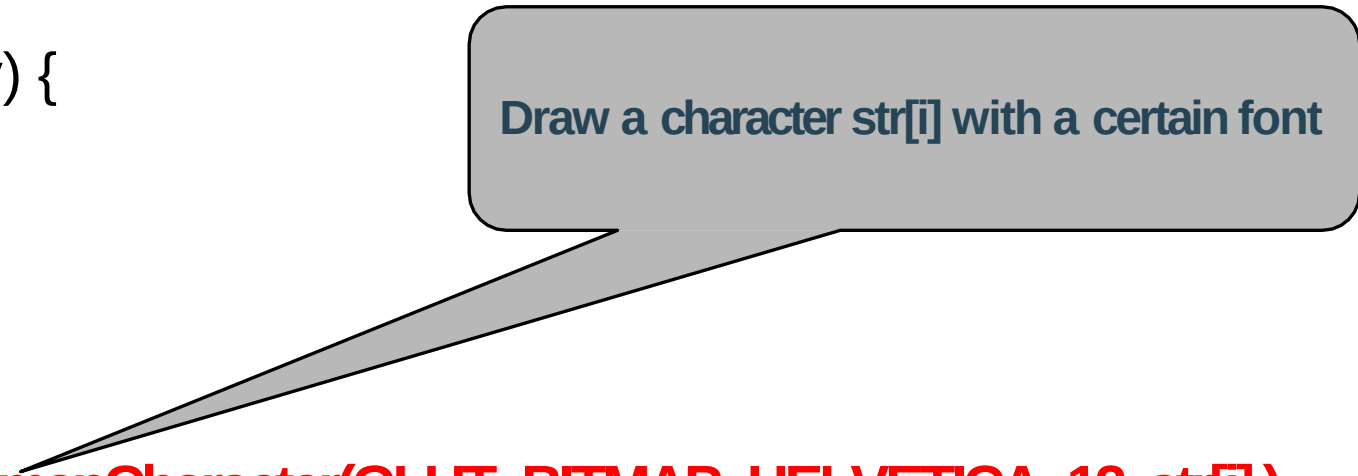
DRAWING TEXT

```
void printSome(char *str,int x,int y) {  
    glColor3f (0, 1.0, 0.0);  
    glRasterPos2d(x,y);  
    for (int i=0;i<strlen(str);i++)  
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, str[i]);  
    glFlush();  
}
```

Indicates the logical location (x,y) where the text is to be drawn

DRAWING TEXT

```
void printSome(char *str,int x,int y) {  
    glColor3f (0, 1.0, 0.0);  
    glRasterPos2d(x,y);  
    for (int i=0;i<strlen(str);i++)  
  
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, str[i] );  
  
    glFlush();  
}
```



Draw a character `str[i]` with a certain font

BITMAP FONTS

There are 7 bitmap fonts

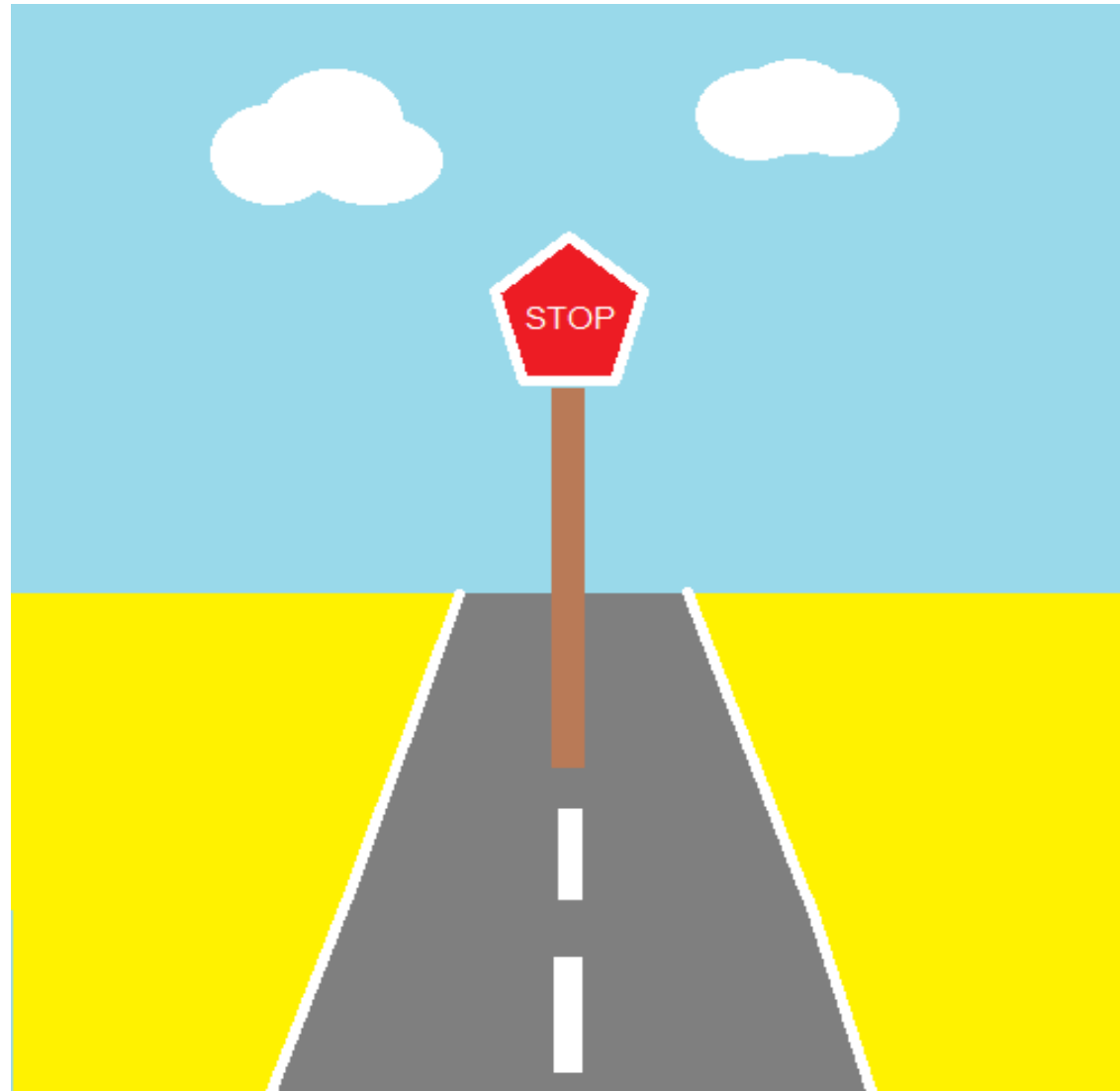
- GLUT_BITMAP_8_BY_13
- GLUT_BITMAP_9_BY_15
- GLUT_BITMAP_TIMES_ROMAN_10
- GLUT_BITMAP_TIMES_ROMAN_24
- GLUT_BITMAP_HELVETICA_10
- GLUT_BITMAP_HELVETICA_12
- GLUT_BITMAP_HELVETICA_18

MAIN()

```
int main( int argc, char ** argv)
{
    glutInit( &argc, argv);
    glutInitDisplayMode( GLUT_DOUBLE |GLUT_RGB);
    glutInitWindowPosition( 100, 100);
    glutInitWindowSize( viewWidth, viewHeight);
    glutCreateWindow( "problem demonstration");
    init();
    glutDisplayFunc( myDisplay);
    glutMainLoop();
}
```

```
void init()
{
    glClearColor( 0.0, 0.0, 1.0, 1.0); // COLOR
    BACKGROUND
    glMatrixMode( GL_PROJECTION);
    gluOrtho2D( 0.0, logWidth, 0.0, logHeight);
}
void myDisplay()
{
    glClear( GL_COLOR_BUFFER_BIT);
    printSome("Hello",centerX,centerY)
    glutSwapBuffers();
}
```

DRAW THE STOP SIGN SCENE



WELCOME SCENE





THANKS